SemFuzz: A Semantics-Aware Fuzzing Framework for Network Protocol Implementations

Yanbang Sun
Tianjin University
Tianjin, China
ybsun@tju.edu.cn

Quan Luo
QI-ANXIN Codesafe Team
Beijing, China
a4651386@163.com

Yuelin Wang
Tianjin University
Tianjin, China
violin0613@tju.edu.cn

Qian Chen
QI-ANXIN Codesafe Team
Beijing, China
cq674350529@gmail.com

Benjin Liu
QI-ANXIN Codesafe Team
Beijing, China
liubenjin@qianxin.com

Ruiqi Chen
QI-ANXIN Codesafe Team
Beijing, China
chenruiqi@qianxin.com

Qing Huang
Jiangxi Normal University
Nanchang, China
qh@whu.edu.cn

Xiaohong Li*
Tianjin University
Tianjin, China
xiaohongli@tju.edu.cn

Junjie Wang*
Tianjin University
Tianjin, China
junjie.wang@tju.edu.cn

Abstract

Network protocols are the foundation of modern communication, yet their implementations often contain semantic vulnerabilities stemming from inadequate understanding of specification semantics. Existing gray-box and black-box testing approaches lack semantic modeling of protocols, making it difficult to precisely express testing intent and cover boundary conditions. Moreover, they typically rely on coarse-grained oracles such as crashes, which are inadequate for identifying deep semantic vulnerabilities. To address these limitations, we present a semantics-aware fuzzing framework, SemFuzz. The framework leverages large language models to extract structured semantic rules from RFC documents and generates test cases that intentionally violate these rules to encode specific testing intents. It then detects deep semantic vulnerabilities by comparing the observed responses with the expected ones. Evaluation on seven widely deployed protocol implementations shows that SemFuzz identified sixteen potential vulnerabilities, ten of which have been confirmed. Among the confirmed vulnerabilities, five were previously unknown and four have been assigned CVEs. These results demonstrate the effectiveness of SemFuzz in detecting semantic vulnerabilities.

CCS Concepts

• **Security and privacy → Web protocol security**.

Keywords

network protocol implementation; large language model; vulnerability detection

* co-corresponding author.

This work is licensed under a Creative Commons Attribution 4.0 International License.

WWW '26, Dubai, United Arab Emirates

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04
https://doi.org/10.1145/3774904.3792541

ACM Reference Format:
Yanbang Sun, Quan Luo, Yuelin Wang, Qian Chen, Benjin Liu, Ruiqi Chen, Qing Huang, Xiaohong Li*, and Junjie Wang*. 2026. SemFuzz: A Semantics-Aware Fuzzing Framework for Network Protocol Implementations. In *Proceedings of the ACM Web Conference 2026 (WWW '26), April 13–17, 2026, Dubai, United Arab Emirates.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3774904.3792541

1 Introduction

Network protocols are the cornerstone of modern communication systems, supporting data exchange across various public services. However, flaws in their implementations often evolve into severe security threats. This issue is particularly critical for closed-source protocol stacks, which are widely deployed in sensitive domains such as government agencies, industrial control systems, and healthcare infrastructures. Once a flaw is introduced, its impact can propagate on a massive scale. For instance, the Windows IPv6 protocol stack (`tcpip.sys`) once exposed the Bad Neighbor vulnerability [9], exposing nearly one billion devices to remote denial-of-service (DoS) attacks. Recent studies [18, 40] further show that such vulnerabilities are increasing in number, underscoring the urgent need for more effective detection techniques.

In general, existing methods fall into two categories: grey-box [22, 24, 31, 32, 35, 41] and black-box testing [6, 12, 29, 36]. Despite the significant success of these methods, they still suffer from two inherent limitations when detecting semantic vulnerabilities in protocol implementations, especially in closed-source systems. First, they lack semantic awareness, making it difficult to generate test cases that effectively cover boundary scenarios. Second, they rely on coarse-grained oracles, which leads to the overlook of many semantic vulnerabilities that do not immediately trigger crashes.

Fig. 1 illustrates a typical example: according to Section 4.2 of Request for Comments (RFC) 8446 [34], when the *pre_shared_key* extension is present in a *ClientHello* message, it must appear as the last extension in the extension list. However, the TLS implementation on the Windows platform (`schannel.dll`) does not strictly adhere to this requirement. An attacker can craft a message with

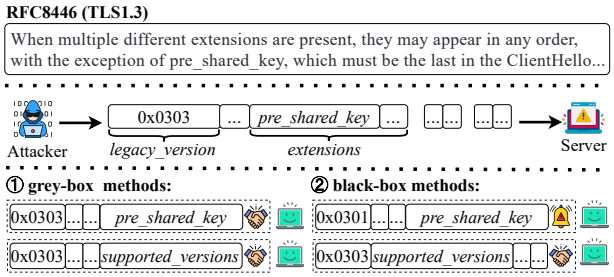


Figure 1: Limitation of existing methods

this extension placed in a non-terminal position and still complete the handshake. After a long period of specific interactions, this behavior may cause the program to crash, leading to a DoS attack.

Existing methods struggle to detect such semantic vulnerabilities. specificity, gray-box approaches (Fig. 1-①) rely on runtime signals such as coverage to drive mutations, but stable binary instrumentation is often infeasible for closed-source protocol stacks [22]; once feedback is lost, gray-box tools degrade into random or grammar-level mutators. Moreover, due to the lack of semantic knowledge, they struggle to construct boundary test cases such as reordering of extension fields, causing vulnerabilities to be missed. Black-box methods (Fig. 1-②) do not depend on instrumentation, but typically only model syntactic structure and likewise lack an understanding of semantics. They can mutate field values (e.g., *legacy_version*) to trigger alerts, but still fail to detect this vulnerability. Even with manually crafted templates generating semantically violating inputs (the second packet in ②), the coarse-grained oracles (e.g., crash) often fail to identify semantic deviations that do not manifest immediately, leading to missed vulnerabilities.

In this paper, we propose SemFuzz, a semantics-aware black-box fuzzing framework for protocol implementations to detect deep semantic vulnerabilities. The core idea is to leverage large language models (LLMs) to extract semantic knowledge from RFC documentation and employ it to guide a closed-loop testing process consisting of semantic modeling, intent-driven mutation, and response verification. Specifically, to compensate for the lack of semantic awareness, SemFuzz introduces semantic modeling, leveraging an LLM to parse RFC documents and extract structured semantic rules. Importantly, each rule specifies both construction constraints, which describe valid message formats, and processing expectations, which define the expected response that these messages should trigger. It converts unstructured natural language descriptions into precise, machine-readable rules, thus establishing a solid semantic foundation for subsequent testing.

Based on the extracted semantic rules, SemFuzz adopts an intent-driven mutation strategy to generate targeted test cases. Specifically, it applies atomic operations (such as adding, deleting, or updating fields) to seed messages captured from real traffic according to the construction constraints of a given semantic rule, thereby generating test inputs that deliberately violate the semantic constraints. This ensures that every test case carries a clear testing intent, aiming to evaluate the behavior of protocol implementations under specific boundary conditions. Finally, to address the oracle problem, we introduce a precise semantic oracle to detect semantic vulnerabilities. For each test case that is designed to violate a specific construction constraint, SemFuzz extracts its corresponding processing expectation from the semantic rule and treats it as the expected response for that test. Our oracle is then defined as follows: if the target's actual response deviates from this expected response, a potential vulnerability is identified. This specification-based response comparison mechanism enables SemFuzz to go beyond traditional crash or memory error detection, thereby systematically uncovering a wide range of critical semantic vulnerabilities.

In the systematic evaluation across seven protocol implementations, SemFuzz demonstrated exceptional vulnerability detection capability, identifying 16 potential vulnerabilities, of which 10 were real vulnerability, achieving a detection accuracy of 62.5%. Through responsible disclosure, 10 vulnerabilities have been confirmed, including 5 previously unknown ones (with 4 CVEs). SemFuzz significantly outperforms existing approaches, with the best baseline detecting only 5 vulnerabilities. Ablation studies demonstrated the effectiveness of the framework's design: the semantic rule constructor improved semantic modeling accuracy by 5.3% and contributed to the discovery of 2 vulnerabilities, while the test case generator significantly enhanced testing effectiveness, increasing test accuracy by 142% and uncovering 8 additional real vulnerabilities. Multi-model experiments also show that SemFuzz's performance stems from its reasonable design rather than any specific LLM.

This paper makes the following contributions:

- **A semantic-aware fuzzing paradigm** is proposed, which leverages LLMs to extract protocol semantics from RFCs, transforming unstructured semantic knowledge into executable testing intents to address existing limitations.
- **A closed-loop workflow** integrating semantic modeling, intent mutation, and response validation is designed, where semantic rules guide the generation of violating test cases and the construction of precise semantic oracles, enabling efficient detection of deep semantic vulnerabilities.
- **A systematic evaluation** across seven widely deployed protocol implementations demonstrates the superior effectiveness of SemFuzz, which discovered 10 real vulnerabilities, including 5 previously unknown ones (4 assigned CVEs).

2 Background and Motivation

2.1 Preliminary

RFCs [3, 14, 21, 34] specify message structures, field constraints, and error-handling procedures, serving as the basis for protocol implementations. However, these specifications are written in natural language for human understanding, making it difficult for existing testing methods to leverage their semantic information directly. To facilitate subsequent semantic modeling, we formalize the RFC specification as a set of requirements:

$$\mathcal{D} = \{\mathcal{R}_1, \ldots, \mathcal{R}_n\}$$

where each $\mathcal{R}_i = (p, m, c)$ represents the protocol name, message type, and specific content.

2.2 Motivation Example

The key insight of our approach is to leverage the semantic knowledge in RFCs to guide the generation of targeted test cases and

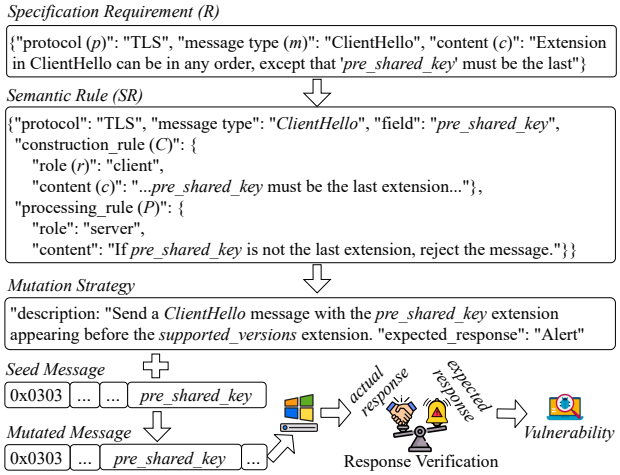


Figure 2: Motivation example

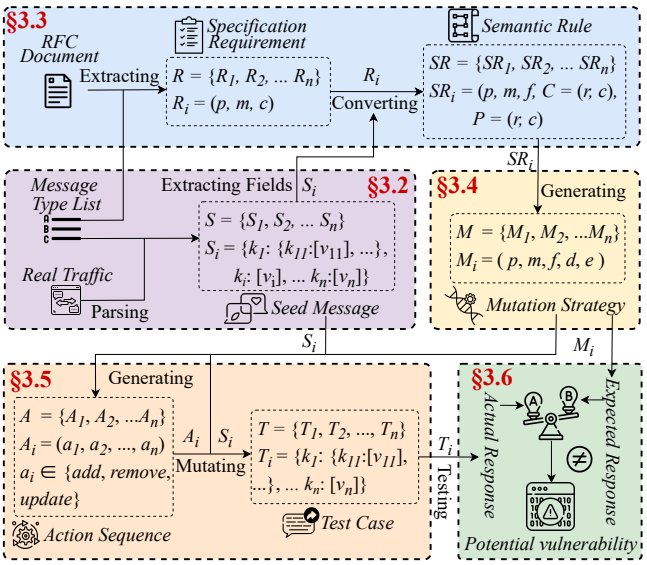


Figure 3: Overall architecture of SemFuzz

to identify potential semantic vulnerabilities by comparing the expected and actual responses. Formally, a specification requirement $\mathcal{R}_i$ can be converted to a semantic rule $\mathcal{SR}$, which written as

$$\mathcal{SR}_i = (p, m, f, C, \mathcal{P})$$

where p, m, and f are the protocol, message, and field names; C is the construction constraint and $\mathcal{P}$ is the processing expectation. The construction constraint is represented as $C = (r, c)$, where r denotes the role (e.g., client) and c represents the specific content. $\mathcal{P}$ follows the same format as C.

The motivation example of our approach is illustrated in Fig. 2. We first extract specification requirements $\mathcal{R}$ from RFC documents and convert them into semantic rules $\mathcal{SR}$ which explicitly define both construction rules (i.e., what the client should send) and processing rules (i.e., how the server should respond). Based on these rules, we generate mutation strategies that intentionally violate specific construction constraints, e.g., placing the *pre_shared_key* extension before the *supported_versions* extension. Each strategy specifies the expected server response (e.g., Alert). By comparing the server's actual response with the expected result, we successfully detected a semantic vulnerability.

3 METHODOLOGY

3.1 Overview

To enable semantic vulnerability detection, we design a five-stage black-box fuzzing framework, SemFuzz (Fig. 3): traffic collector (§3.2), structured rule constructor (§3.3), mutation-strategy generator (§3.4), test-case generator (§3.5), and response verifier (§3.6).

Specifically, as shown in Fig. 3, traffic collector begins with the collection of real-world traffic based on the given message type list ($\mathcal{L}$), from which we construct a set $\mathcal{S}$ of seed messages:

$$\mathcal{S} = \{\mathcal{S}_1, \mathcal{S}_2, \ldots, \mathcal{S}_n\}$$

Each seed $\mathcal{S}_i$ is represented as a mapping from field keys to values: $\mathcal{S}_i = (k_1 : [v_1], \ldots, k_n : [v_n])$, where k_i denotes a message field and v_i are observed values for that field. Meanwhile, the structured rule constructor uses a LLM to parse the RFC text, extract requirements $\mathcal{R}$, and further convert them into semantic rule $\mathcal{SR}$.

For a selected $\mathcal{SR}_i$, the mutation strategy generator generates one or more candidate mutation strategies:

$$\mathcal{M} = \{\mathcal{M}_1, \mathcal{M}_2, \ldots, \mathcal{M}_n\}$$

where each $\mathcal{M}_i = (p, m, f, d, e)$ encodes a specific strategy to violate the construction rule C (ref. §2.2). Here, d is a textual description of the mutation strategy, derived from the violation of C; e denotes the expected behavior upon violation, e.g., Alert.

Based on the mutation strategies $\mathcal{M}$ and the seed message $\mathcal{S}$, SemFuzz then generates a set of action sequences:

$$\mathcal{A} = \{\mathcal{A}_1, \mathcal{A}_2, \ldots, \mathcal{A}_n\}$$

The action sequence $\mathcal{A}_i$ consists of a series of atomic action, such as `add`, `remove`, or `update`. Applying $\mathcal{A}_i$ to the corresponding seed message ($\mathcal{S}_i$) then yields a set of test cases:

$$\mathcal{T} = \{\mathcal{T}_1, \mathcal{T}_2, \ldots, \mathcal{T}_n\}$$

Each test case $\mathcal{T}_i$ maintains syntactic validity while intentionally violating the construction rule C. Finally, the set of test cases $\mathcal{T}$ is sent to the target protocol implementation. For each $\mathcal{T}_i$, we record its actual response (r) and compare it with the expected response (e) of the mutation strategy $\mathcal{M}_i$. If the two are inconsistent, the implementation is considered to have a potential vulnerability.

3.2 Traffic Collector

To ensure the realism and validity of test inputs, we design a traffic collector to automatically extract real-world seed messages $\mathcal{S}$ from network environments. The traffic collection process begins with specifying the message type and the IP address of the target application. We then trigger interactions between the client and the server by executing predefined scripts, thereby stimulating message exchanges. These interactions are captured in real time

Algorithm 1 Structured Rule Construction

Input: RFC document $\mathcal{D}$, message type list $\mathcal{L}$, seed messages $\mathcal{S}$
Output: Structured semantic rules $\mathcal{SR} = \{\mathcal{SR}_1, ..., \mathcal{SR}_n\}$

1: $Text \leftarrow$ `CleanDocument`($\mathcal{D}$)
2: $Paras \leftarrow$ `SplitIntoParagraphs`($Text$)
3: $\mathcal{R} \leftarrow \emptyset$
4: **for all** $para \in Paras$ **do**
5: $(p, m, c) \leftarrow$ `LLM_IdentifySR`($para, \mathcal{L}$)
6: $\mathcal{R} \leftarrow \mathcal{R} \cup \{(p, m, c)\}$
7: **end for**
8: $\mathcal{SR} \leftarrow \emptyset$
9: **for all** $\mathcal{R}_i \in \mathcal{R}$ **do**
10: $\mathcal{F} \leftarrow$ `ExtractFieldPaths`($\mathcal{S}_i$)
11: $(f,\ C,\ \mathcal{P}) \leftarrow$ `LLM_Complete`($\mathcal{R}_i,\ \mathcal{F}$)
12: $\mathcal{SR} \leftarrow \mathcal{SR} \cup (p,\ m,\ f,\ C,\ \mathcal{P})$
13: **end for**
14: **return** $\mathcal{SR}$

using Wireshark [39], and parsed into structured records via its command-line interface, tshark. Each parsed message is treated as a seed $\mathcal{S}_i$ and added to the seed set $\mathcal{S}$ for subsequent mutation.

3.3 Semantic Rule Constructor

To achieve semantic modeling, we introduce the structured rule constructor. The overall procedure is summarized in Algorithm 1, the process involves three main steps: data preprocessing, SR identification, and rule completion. Specifically, in the preprocessing stage (lines 1–2), we first clean the raw document by removing non-essential content, and then segment the remaining text into paragraphs to preserve contextual coherence. In the specification identification stage (lines 3–6), for each paragraph, a LLM is employed to extract a specification item $\mathcal{R}_i$. All identified specification items are aggregated into the set $\mathcal{R}$. In the rule completion stage (lines 8–12), based on each specification $\mathcal{R}_i$, we further derive its corresponding construction rule C and processing rule $\mathcal{P}$, thereby forming a structured rule $\mathcal{SR}_i$. Finally, all structured rules constitute the semantic rule set $\mathcal{SR}$, which provides a unified semantic foundation for subsequent analysis and verification.

3.3.1 Data preprocessing. To ensure accurate extraction of semantic rules, we preprocess each RFC document according to its identifier. RFCs typically contain non-essential sections such as the table of contents, copyright notices, and references that are irrelevant to protocol semantics. We retain only the main body of the document, specifically the portion between the table of contents and the references, where most protocol definitions and operational details reside. The remaining content is divided into coherent paragraphs to reduce LLM inference complexity and enable accurate rule extraction. To preserve contextual relevance, each paragraph is prefixed with its hierarchical chapter titles, reflecting the original document structure. Finally, each paragraph is represented as a key–value pair in the form `chapter path`: `paragraph text`.

3.3.2 Specification Identification. We design a specification identification module that leverages LLMs to analyze paragraph-level content and extract semantic knowledge from natural language descriptions. Given a paragraph and a predefined message type set $\mathcal{L}$, the module identifies specification requirements $\mathcal{R} = (p, m, c)$, which characterize the normative behaviors of either the client or the server and serve as the foundation for subsequent semantic rule completion. More details about the prompt design in this paper can be found in the Section A of the Appendix.

3.3.3 Rule Completion. The rule completion module transforms each extracted specification $\mathcal{R}_i$ into a structured semantic rule $\mathcal{SR}_i = (p, m, f, C, \mathcal{P})$. Given the specification content c and the corresponding seed message $\mathcal{S}_i$, the LLM leverages the message structure to identify the relevant field f and derive two complementary components: the construction rule C and the processing rule $\mathcal{P}$. This process converts textual specifications into explicit semantic constraints and behavioral descriptions, forming a consistent rule set $\mathcal{SR}$ for downstream mutation and verification tasks.

3.4 Mutation Strategy Generator

The mutation strategy generator transforms semantic rules into executable testing strategies that intentionally violate protocol constraints. Given a semantic rule $\mathcal{SR}_i = (p, m, f, C, \mathcal{P})$, the module employs a LLM to generate one or more mutation strategies $M = (p, m, f, d, e)$, where d describes how to deviate from the construction rule C, and e represents the expected response inferred from the processing rule $\mathcal{P}$. By converting compliant semantics into explicit violation cases, this process enables goal-directed fuzzing that systematically explores protocol boundaries and uncovers potential implementation flaws. To ensure consistency in response evaluation, the LLM output is constrained to two high-level feedback categories: normal feedback (e.g., successful handshake) and error feedback (e.g., no response or abnormal termination). More details can be found in the Appendix.

3.5 Test Case Generator

To accurately translate semantic intent into targeted test cases, we design a test case generator that applies structured mutation actions to real seed messages, ensuring that the generated cases remain valid while precisely targeting semantic boundary conditions. As shown in Algorithm 2, the generation process consists of two phases: action sequence generation and message mutation.

Specifically, for each mutation strategy $\mathcal{M}_i$ and its corresponding seed message $\mathcal{S}_i$, the module extracts the message structure $\mathcal{F}_i$, retaining only field paths (i.e., field names k) to provide contextual information (line 3). Then, combining the mutation strategy and the message structure, the LLM generates an action sequence $\mathcal{A}_i$ (line 4), where each action is selected from a predefined set of atomic actions, `Add`, `Remove`, and `Update` (see Section 3.5.1), to describe specific mutation behaviors.

In the message mutation stage (line 5), the execution engine applies the generated actions to the seed message to construct the test case $\mathcal{T}_i$. When an action contains incomplete parameters (e.g., missing length fields), the module automatically infers and fills them based on contextual information to ensure syntactic consistency. All generated test cases are collected into the set $\mathcal{T}$, which serves as the input for subsequent response verification (line 6).

Algorithm 2 Test Case Generation

Input: Mutation strategies $\mathcal{M} = \{\mathcal{M}_1, ..., \mathcal{M}_n\}$, seed messages $\mathcal{S} = \{\mathcal{S}_1, ..., \mathcal{S}_n\}$
Output: Test case set $\mathcal{T} = \{\mathcal{T}_1, \mathcal{T}_2, \ldots, \mathcal{T}_n\}$

$\mathcal{T} \leftarrow \emptyset$
for $i = 1$ **to** n **do**
  $\mathcal{F}_i \leftarrow$ `ExtractFieldPaths`$(\mathcal{S}_i)$
  $\mathcal{A}_i \leftarrow$ `LLM_GenerateActions`$(\mathcal{M}_i, \mathcal{F}_i)$
  $\mathcal{T}_i \leftarrow$ `MessageMutation`$(\mathcal{S}_i, \mathcal{A}_i)$
  $\mathcal{T} \leftarrow \mathcal{T} \cup \{\mathcal{T}_i\}$
end for
return $\mathcal{T}$

3.5.1 Action Sequence Generation. Directly leveraging an LLM to modify hexadecimal message data often leads to invalid test cases. This is particularly problematic in binary protocols such as TLS, where small changes in one field (e.g., an extension) may require cascading updates to other dependent fields (e.g., length indicators). To address this, we design the action sequence generation module, which transforms each mutation strategy into a high-level sequence of atomic actions. Instead of directly manipulating raw bytes, the module specifies how to mutate a message in an abstract, structured manner. The generated action sequence is then executed by a deterministic mutation engine that ensures structural integrity and automatically fills in undefined values.

Specifically, the module takes as input a mutation strategy $\mathcal{M}_i$ and the corresponding message structure $\mathcal{F}_i$ extracted from the seed message. Each action belongs to one of the following types:

- `add(fields, position, value)`: Insert a field at the specified position; if the position is empty, the field is added to the end.
- `remove(fields)`: removes a field or subtree from the message based on its field path.
- `update(fields, new_value)`: updates the value of a field; if it is empty, it is inferred during execution.

This design allows the LLM to focus on semantic-level mutation, while the execution engine maintains structural consistency, thereby bridging symbolic reasoning and byte-level manipulation.

3.5.2 Message Mutation. To ensure correctness and maintain syntactic validity, the execution of mutation actions is delegated to a deterministic message mutation engine. This component addresses the shortcomings of LLMs in byte-level accuracy and length-dependent computations. The mutation engine takes as input a seed message and the corresponding action sequence. For each action, it invokes predefined transformation functions to apply the requested modification to the nested message structure. This structured representation preserves field hierarchy and allows precise updates. When the action sequence contains undefined parameters, such as empty values in an `update` action, the mutation engine infers the missing content using the message context. For instance, if an extension is inserted, the engine computes and updates the associated *extension_len* field accordingly.

3.6 Response Verifier

To complete the fuzzing feedback loop, we design the response verifier to evaluate whether the implementation's actual response aligns with the expected response e defined in the semantic rule. This closes the gap between mutation intent and response verification, enabling automated inconsistency detection. During execution, the test case is sent by the role specified in the construction rule C, and the response is observed from the corresponding receiver role. The verifier monitors the interaction and records the implementation's actual response. If any discrepancy is observed between the two, it is labeled as a potential vulnerability.

4 Evaluation

To evaluate the effectiveness of SemFuzz, we aim to answer the following research questions:

RQ1: How effective is the LLM-based semantic modeling?
RQ2: How effective is SemFuzz in detecting vulnerabilities compared to existing methods?
RQ3: How does each module in SemFuzz contribute to the overall effectiveness of fuzzing?
RQ4: How does the choice of underlying LLM affect the overall fuzzing performance of SemFuzz?

4.1 Experimental Setup

4.1.1 Baselines. To evaluate the effectiveness of SemFuzz, we selected four representative existing methods for comparison. Among the grey-box methods, ChatAFL [24] leverages the protocol syntax knowledge embedded in LLMs to guide fuzzing but is only applicable to plaintext protocols (e.g., HTTP). On the other hand, Fuzztruction-Net [5] generates test cases by performing fault injection on one side of the communication. However, this method relies on compile-time instrumentation of source code and is therefore applicable only to open-source protocol implementations.

Among the black-box methods, Hdiff [36] detects inconsistencies between protocol implementations through differential testing, but its effectiveness heavily depends on predefined templates. In contrast, Bleem [22] infers states and guides testing by analyzing external network traffic, making it applicable to both open-source and closed-source protocol implementations.

In addition, to evaluate the contribution of each module in the framework, we designed two ablation variants. First, to assess the role of the specification identification module in structured semantic modeling, we designed w/oSI, which directly extracts semantic rules from text. Second, to evaluate the effect of action sequence generation, we designed $w/oAction$, which allows the large language model to directly modify seed messages.

4.1.2 Benchmark dataset. To objectively evaluate semantic artifacts, we constructed a benchmark dataset by systematically annotating the core RFC specifications of TLS 1.3, HTTP/1.1, IPv6, and DNS. Eight cybersecurity master students independently cross-annotated each document, followed by expert adjudication to resolve disagreements and ensure semantic consistency. The two-stage annotation process first extracted normative requirements from RFC text and then transformed them into structured semantic rules. In total, the 28-hour effort yielded 1,721 high-quality semantic rules, comprising 624 from TLS 1.3, 540 from HTTP/1.1, 298 from IPv6, and 259 from DNS.

Table 1: Details of our testing targets (*server version)

Target	Version	Protocol	RFC	Message Type
Dns.exe	*.143/*.206	DNS	1035	Binary
Tcpip.sys	*.169/*.351	IPv6	8200	Binary
Schannel.dll	*.143/*.103	TLS 1.3	8446	Binary
Http.sys	*.143/*.351	HTTP/1.1	7230	Text
Openssl	3.1.3	TLS 1.3	8446	Binary
LibreSSL	3.4.0	TLS 1.3	8446	Binary
Nginx	1.27.2	HTTP/1.1	7230	Text

4.1.3 Experimental Environment. All experiments were conducted on a high-performance workstation equipped with two 18-core 2.3 GHz Intel Xeon E5-2699 CPUs and 32 GB of memory. Three virtualized environments were configured on this workstation to cover both closed-source and open-source network protocol stack implementations. Specifically, the Windows test environment includes Windows Server 2022 (v20348) and Windows Server 2016 (v14393), used to evaluate the native network protocol components: `dns.exe` (DNS), `tcpip.sys` (IPv6), `schannel.dll` (TLS 1.3), and `http.sys` (HTTP/1.1), as listed in Table 1 (Rows 2–5). In addition, we deployed three widely used open-source implementations on a Linux virtual machine running Ubuntu 20.04: `OpenSSL` and `LibreSSL` (for TLS 1.3), and `Nginx` (for HTTP/1.1), as shown in Table 1 (Rows 6–8). All virtual machines share identical hardware configurations to ensure the fairness and comparability of experimental results. All baselines were run under the same hardware configuration (CPU/RAM) and the same initial seeds, with 24 hours of exploration time.

SemFuzz is based on the GPT-4o model [26]. To control the randomness in LLM responses, we conducted multiple experiments under different parameter settings. Based on empirical results, we selected the optimal configuration: the parameters top_p and temperature were set to 0.1 and 0.5, respectively. Since ChatAFL is implemented based on AFLNet [31], we replaced it with NetAFL [16] when conducting experiments in the Windows environment. To ensure a fair comparison, the base model of ChatAFL was also switched from GPT-3.5 to GPT-4o. For Fuzztruction-Net [5], we followed its default configuration. Specifically, when testing OpenSSL and LibreSSL servers, the method employed their built-in client tools (e.g., s_client) as the "weird peers" to generate test inputs, while testing the Nginx server used the client provided by the ngtcp2 project as the "weird peer".

4.1.4 Evaluation Metrics. We use precision, recall, and f1-score to evaluate the performance of rule extraction ($\mathcal{R}$) and transformation ($\mathcal{SR}$). A generated rule is considered correct if it semantically matches an entry in the manually constructed benchmark, with strict consistency required for message and field names. For downstream tasks, accuracy is used as the evaluation metric. Due to the one-to-many nature of mutation strategy ($\mathcal{M}$) generation, it is impractical to build a complete benchmark; therefore, correctness is determined manually. A strategy is regarded as correct if it effectively violates the corresponding rule and produces a verifiable response. The accuracy of test cases ($\mathcal{T}$) is determined by their syntactic validity; a test case is deemed correct if it can be properly encoded and successfully sent.

Table 2: Semantic modeling evaluation results

RFC	Paras.	$\mathcal{R}$			$\mathcal{SR}$			$\mathcal{M}$		$\mathcal{T}$	
		P	R	F1	P	R	F1	Correct	Acc.	Correct	Acc.
1035	280	0.78	0.86	0.82	0.74	0.82	0.78	960	0.93	929	0.90
8200	160	0.83	0.89	0.86	0.78	0.84	0.81	856	0.94	828	0.91
8446	521	0.82	0.87	0.84	0.80	0.85	0.82	1,997	0.91	1,728	0.79
7230	357	0.79	0.89	0.84	0.76	0.86	0.81	1,632	0.90	1,599	0.88
Average		0.80	0.88	0.84	0.77	0.84	0.80	1,361	0.92	1,271	0.87

4.2 RQ1: Effectiveness of Semantic Modeling

Table 2 presents the detailed results of our semantic modeling process. After processing all 1,318 RFC paragraphs, our framework demonstrates strong performance in both core stages: identifying specification requirements ($\mathcal{R}$) and transforming them into semantic rules ($\mathcal{SR}$). The average F1-scores reach 0.84 and 0.80, respectively, indicating that SemFuzz can accurately extract and structure protocol semantics from unstructured text. The high-quality semantic rules provide a solid foundation for subsequent tasks. Based on these rules, the downstream modules successfully generate 5,940 mutation strategies ($\mathcal{M}$) and test cases ($\mathcal{T}$), achieving average accuracies of 0.92 and 0.87, respectively. This provides strong evidence of the effectiveness of our semantic modeling approach.

We also observe performance variations across protocols. In particular, for TLS 1.3 (RFC 8446), the accuracy of test case generation drops to 0.79, which is notably lower than that of other protocols (all above 0.90). This decrease mainly results from the structural complexity of TLS messages such as *ClientHello*, whose deeply nested fields pose greater challenges to the reasoning capability of large language models, occasionally leading to mutation strategies that reference nonexistent fields. Despite this variation, the overall performance of our framework remains strong, confirming the effectiveness of our approach in semantic rule modeling.

> ***Answer to RQ1**: Our framework demonstrates high accuracy and F1-scores across all stages of semantic modeling. The average F1-score remains no lower than 80%, and the average accuracy exceeds 87%, proving its effectiveness in semantic modeling and its strong capability to support semantics-aware fuzzing.*

4.3 RQ2: Comparison with Existing Methods

SemFuzz discovered a total of 16 potential vulnerabilities, among which 10 were confirmed by developers as real vulnerabilities, resulting in an accuracy of 62.5%. Table 4 shows the number of real vulnerabilities detected by baseline methods: BLEEM (5), ChatAFL (1), Hdiff (2), and Fuzztruction-Net (0). Among the 11 unique vulnerabilities collectively discovered by all tools, SemFuzz covered 10 of them, demonstrating the best performance.

For gray-box methods, Fuzztruction-Net [5] relies on fault injection through source-code instrumentation, making it inapplicable to closed-source targets (e.g., `dns.exe`, `tcpip.sys`). Moreover, due to its lack of semantic awareness, it fails to generate targeted test cases and did not discover any vulnerabilities even on open-source targets. ChatAFL [24], although leveraging LLMs, is mainly designed for text-based protocols and cannot handle binary protocols such as DNS, IPv6, and TLS. Furthermore, it lacks explicit semantic modeling, discovering only one vulnerability in `http.sys`.

Table 3: Overview of the 10 confirmed vulnerabilities we found in different targets

Target	ID	Type	Description	Status	CVE
Dns.exe	1	Cache pollution	Insufficient validation of unsolicited records in DNS responses.	Fixed	-
Tcpip.sys	2	Integer overflow	Integer overflow from an unrecognized Option Type in Destination Options header.	Fixed	CVE-2021-24074
	3	Buffer overflow	Buffer overflow from incorrect ESP header placement before the Fragment header.	Fixed	CVE-2022-34718
	4	Integer overflow	Integer overflow from parsing a malformed Destination Options header.	Fixed	-
Schannel.dll	5	Uaf	Missing position check for the pre_shared_key extension in ClientHello.	Fixed	CVE-2023-28233
	6	Uaf	Missing position check for the pre_shared_key extension in ClientHello.	Fixed	CVE-2023-28234
Http.sys	7	Tampering	Improper handling of extra whitespace between header field-name and colon.	Fixed	-
	8	Uaf	UAF caused by parsing random values in the Accept-Encoding header.	Fixed	-
	9	Buffer overflow	Improper handling of HTTP Trailers.	Fixed	-
	10	Integer overflow	Integer overflow from missing length validation on an excessively long ServiceName.	Fixed	-

Note: #2 and #3 were discovered by an earlier version of SemFuzz (rule-based modeling), while the current version achieves the same results via LLM-based automated modeling.

Table 4: Unique vulnerabilities found by different methods

Target	ChatAFL	Fuzztruction-Net	Hdiff	BLEEM	SemFuzz	Unique Vuls.
Dns.exe	-	-	-	0	1	1
Tcpip.sys	-	-	-	1	3	3
Schannel.dll	-	-	-	0	2	2
Http.sys	1	-	2	3	4	4
OpenSSL	-	0	-	0	0	0
LibreSSL	-	0	-	1	0	1
Nginx	0	0	-	0	0	0
Sum	**1**	**0**	**2**	**5**	**10**	**11**

For black-box methods, Hdiff [36] is specifically designed for the HTTP protocol and thus exhibits poor extensibility to other protocols. Moreover, its heavy reliance on manually predefined templates limits its ability to explore boundary conditions, resulting in only two discovered vulnerabilities. BLEEM [22] demonstrates relatively strong performance and detects 5 vulnerabilities. Notably, it detected a unique vulnerability in LibreSSL. This vulnerability involves memory corruption within the X.509 certificate parsing module, whose behavior is specified by independent standards such as RFC 5280. Since our work focuses on modeling the semantics of the TLS 1.3 protocol based on RFC 8446, we did not detect this vulnerability. Nevertheless, BLEEM lacks a deep understanding of RFC specifications, and its random mutation and state exploration strategies struggle to cover complex boundary conditions. Moreover, its reliance on coarse-grained oracles (e.g., server crashes) further limits its performance, causing it to miss 5 vulnerabilities.

Table 3 presents the detailed information of the 10 real vulnerabilities discovered by SemFuzz. These vulnerabilities have been confirmed by vendors, including 5 previously unknown vulnerabilities (#1, #2, #3, #5, #6), for which 4 CVEs have been assigned. In addition, 6 potential vulnerabilities were confirmed to be false positives: one caused by an incorrect action sequence generation, one triggered by an incorrect mutation strategy, two due to inconsistencies in the experimental configuration, and the remaining two were deemed non-vulnerabilities by developers because they were difficult to exploit. More details can be found in the appendix.

> ***Answer to RQ2****: SemFuzz successfully discovered 10 vulnerabilities, including 5 previously unknown ones (with 4 CVEs). In contrast, the best baseline detected only 5 vulnerabilities.*

Table 5: Effectiveness of each module in SemFuzz

Protocol	SemFuzz			$^*w/oSI$			SemFuzz		$^*w/oAction$	
	P	R	F1	P	R	F1	Correct	Acc.	Correct	Acc.
DNS	0.74	0.82	0.78	0.69	0.79	0.74	929	0.90	539	0.52
IPv6	0.78	0.84	0.81	0.75	0.78	0.77	828	0.91	198	0.22
TLS 1.3	0.80	0.85	0.82	0.73	0.79	0.76	1,728	0.79	134	0.06
HTTP/1.1	0.76	0.86	0.81	0.73	0.83	0.78	1,599	0.88	1,263	0.70
Average	0.77	0.84	0.80	0.73	0.80	0.76	1,271	0.87	534	0.36

Table 6: Ablation experiment results

Target	SemFuzz			$^*w/oSR$			$^*w/oAction$		
	PVul.	RVul.	Acc.	PVul.	RVul.	Acc.	PVul.	RVul.	Acc.
Dns.exe	2	1	0.50	2	1	0.50	0	0	-
Tcpip.sys	4	3	0.75	3	2	0.67	0	0	-
Schannel.dll	3	2	0.67	3	2	0.67	0	0	-
Http.sys	4	4	1.00	3	3	1.00	4	2	0.5
Openssl	3	0	-	3	0	-	0	0	-
Sum	16	10	0.63	14	8	0.57	4	2	0.5

4.4 RQ3: Ablation Study

To evaluate the effectiveness of each core module in SemFuzz, we conducted a series of ablation studies. First, the specification identification module significantly improves the quality of semantic rule extraction from RFC documents. As shown in Table 5, SemFuzz achieves an average precision of 77%, recall of 84%, and F1-score of 80% in the semantic rule extraction. In contrast, the variant without this module ($^*w/oSI$) attains an F1-score of 76%. This demonstrates that the module effectively filters irrelevant text and provides precise inputs for subsequent structured modeling, thereby enhancing the overall performance of semantic rule extraction.

Second, the action sequence generation module has a decisive impact on the quality of test case generation. The results show that SemFuzz generates an average of 1,271 correct test cases with an accuracy of 87%. When this module is disabled ($^*w/oAction$), the system produces only 534 correct cases, and the accuracy drops sharply to 36%. The performance gap becomes more pronounced for complex protocols such as IPv6 and TLS 1.3, where accuracy falls to 22% and 6%, respectively. These results indicate that, without high-level action guidance, the LLM struggles to precisely locate specific fields and generate consistent byte-level mutations.

Finally, Table 6 presents the contribution of each module to vulnerability detection. SemFuzz identifies 16 potential vulnerabilities,

Table 7: The impact of the LLM on overall performance

Target	SemFuzz (GPT-4o)			*GPT-5			*Gemini-1.5 Pro			*Gemini-2.5 Pro		
	PVul.	RVul.	Acc.	PVul.	RVul.	Acc.	PVul.	RVul.	Acc.	PVul.	RVul.	Acc.
Dns.exe	2	1	0.50	1	1	1.00	1	0	-	1	1	1.00
Tcpip.sys	4	3	0.75	4	3	0.75	5	3	0.60	4	3	0.75
Schannel.dll	3	2	0.67	3	2	0.67	4	2	0.50	3	2	0.67
Http.sys	4	4	1.00	4	4	1.00	4	4	1.00	4	4	1.00
Openssl	3	0	-	3	0	-	4	0	-	3	0	-
Sum	16	10	0.63	15	10	0.67	18	9	0.50	15	10	0.67

10 of which are real vulnerabilities. When the specification identification module is removed (*w/oSI), the number of confirmed vulnerabilities drops to 8, demonstrating the importance of fine-grained semantic modeling for vulnerability detection. Moreover, disabling the action sequence generation (*$w/oAction$) severely degrades fuzzing effectiveness, yielding only 2 real vulnerabilities. This result verifies the essential role of action sequences in translating mutation strategies into executable test cases.

> ***Answer to RQ3****: The core modules of SemFuzz enhance the overall performance. The semantic rule constructor improves semantic modeling performance by approximately 5.3% and detects 2 more vulnerabilities. The test case generator further increases test case accuracy by 142% and detects 8 more vulnerabilities.*

4.5 RQ4: The Impact of Models on Fuzzing

In this RQ, we replace the LLM GPT-4o with GPT-5 [27], Gemini-1.5 Pro [10], and Gemini-2.5 Pro [11], and conduct experiments under the same settings. As shown in Table 7, despite using different LLMs, all variants were able to discover at least 9 real vulnerabilities. This indicates that the effectiveness of SemFuzz primarily stems from its closed-loop and semantics-aware framework design, rather than relying on the unique capabilities of any specific model. Moreover, even earlier models such as GPT-4o can cover most critical boundary conditions under the guidance of the framework.

The results also reveal performance differences caused by varying reasoning capabilities. For example, Gemini-1.5 Pro detected the largest number of potential vulnerabilities (18), but achieved the lowest precision (50%), as it tends to adopt more aggressive mutation strategies. As the reasoning ability of the LLMs improves, the precision of vulnerability detection increases accordingly. Gemini-2.5 Pro achieved the highest precision at 67%.

> ***Answer to RQ4****: The performance of SemFuzz primarily originates from its closed-loop and semantics-aware design rather than from any specific LLM; stronger reasoning capabilities can further enhance its detection effectiveness.*

5 Threat to Validity

Message-type Selection and Seed-traffic Collection. The effectiveness of the framework depends on two user-provided inputs: a list of protocol message types to test and corresponding seed traffic. In this experiment we manually consulted the IANA registry to select 44 key message types across four protocols; this process took approximately 20 minutes. More details can be found in the Section B of the Appendix. We then captured seed messages from standards-compliant client–server interactions to ensure precise scope control and syntactic correctness of the seed messages.

Evaluation Metric Setup. We did not use traditional fuzzing metrics such as code coverage because SemFuzz is designed for black-box scenarios and targets closed-source protocol implementations (e.g., tcpip.sys), for which instrumentation to obtain coverage is typically infeasible. Instead, we focus on a more direct success criterion: the ability to discover real, verifiable vulnerabilities. This vulnerability-centric evaluation both aligns with the ultimate goals of security testing and directly measures the framework's ability to uncover deep semantic vulnerabilities.

6 Related Work

Recent research on protocol fuzzing can be broadly categorized into grey-box and black-box approaches. Grey-box methods [2, 4, 5, 20, 24, 25, 30, 32, 41] leverage instrumentation to infer protocol states and guide test generation. AFLNet [30], for instance, combines response-code inference with coverage feedback, while ChatAFL [24] exploits LLM-derived syntax knowledge for text-based protocols. Other techniques [5] inject faults into communication peers. However, their dependence on source-level instrumentation limits applicability to closed-source targets such as *tcpip.sys*.

Black-box fuzzers [6, 12, 15, 19, 22, 33, 36] avoid instrumentation and thus suit closed-source settings. BLEEM [22] builds response-based state graphs, and Snipuzz [15] adopts response-aware mutation. However, lacking semantic guidance, they struggle to generate intent-driven inputs for boundary behaviors and rely on crashes as coarse oracles. HDiff [36] uses RFC-guided differential testing to detect semantic divergences in HTTP, but relies on manual templates and offers limited semantic coverage.

Rule extraction from RFCs has been explored for security analysis and test automation [1, 7, 8, 13, 17, 23, 28, 37, 38]. For example, some studies [1, 28] attempt to automatically extract finite state machines (FSMs) from specifications to support automated security analysis. In addition, mGPTFuzz [23] and LLMIF [38] leverage LLMs to infer message structures of specific protocols (e.g., Zigbee) to guide testing. However, these approaches lack fine-grained semantic modeling (such as field-level constraints), limiting their ability to uncover semantic vulnerabilities.

7 Conclusion

This paper proposes a semantic-aware black-box testing framework, SemFuzz. The framework leverages LLMs to extract structured semantic rules from RFC documents. These rules not only guide the generation of intentional violation test cases but also identify precise semantic oracles, thereby addressing the limitations of existing approaches in semantic modeling, intent expression, and semantic vulnerability detection. In the evaluation on seven widely used protocol implementations, SemFuzz discovered 16 potential vulnerabilities, 10 of which were confirmed, including 5 previously unknown vulnerabilities (with 4 assigned CVEs), demonstrating the effectiveness of our framework.

8 Acknowledgments

This work was partly supported by the national key research and development program of china (2023YFB3107100).

References

[1] Abdullah Al Ishtiaq, Sarkar Snigdha Sarathi Das, Syed Md Mukit Rashid, Ali Ranjbar, Kai Tu, Tianwei Wu, Zhezheng Song, Weixuan Wang, Mujtahid Akon, Rui Zhang, et al. 2024. Hermes: Unlocking security analysis of cellular network protocols by synthesizing finite state machines from natural language specifications. In 33rd USENIX Security Symposium (USENIX Security 24). 4445–4462.

[2] Anastasios Andronidis and Cristian Cadar. 2022. SnapFuzz: high-throughput fuzzing of network applications. In Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (Virtual, South Korea) (ISSTA 2022). Association for Computing Machinery, New York, NY, USA, 340–351. doi:10.1145/3533767.3534376

[3] Roy Arends, Rob Austein, Matt Larson, Dan Massey, and Scott Rose. 2005. Protocol modifications for the DNS security extensions.

[4] Jinsheng Ba, Marcel Böhme, Zahra Mirzamomen, and Abhik Roychoudhury. 2022. Stateful Greybox Fuzzing. In 31st USENIX Security Symposium (USENIX Security 22). USENIX Association, Boston, MA, 3255–3272. https://www.usenix.org/conference/usenixsecurity22/presentation/ba

[5] Nils Bars, Moritz Schloegel, Tobias Scharnowski, Nico Schiller, and Thorsten Holz. 2023. Fuzztruction: Using Fault Injection-based Fuzzing to Leverage Implicit Domain Knowledge. In 32nd USENIX Security Symposium (USENIX Security 23). USENIX Association, Anaheim, CA, 1847–1864. https://www.usenix.org/conference/usenixsecurity23/presentation/bars

[6] Chu Chen, Cong Tian, Zhenhua Duan, and Liang Zhao. 2018. RFC-directed differential testing of certificate validation in SSL/TLS implementations. In Proceedings of the 40th International Conference on Software Engineering. 859–870.

[7] Jingting Chen, Feng Li, Qingfang Chen, Ping Li, Lili Xu, and Wei Huo. 2023. EBugDec: Detecting Inconsistency Bugs caused by RFC Evolution in Protocol Implementations. In Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses. 412–425.

[8] Jingting Chen, Feng Li, Mingjie Xu, Jianhua Zhou, and Wei Huo. 2022. RIBDetector: an RFC-guided Inconsistency Bug Detecting Approach for Protocol Implementations. https://api.semanticscholar.org/CorpusID:250976152. 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER) (2022), 641–651.

[9] Microsoft Corporation. 2020. Bad Neighbor. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2020-16898.

[10] Google DeepMind. 2024. Gemini 1.5 Pro. https://blog.google/technology/ai/google-gemini-next-generation-model-february-2024/.

[11] Google DeepMind. 2025. Gemini 2.5 Pro. https://deepmind.google/models/gemini/pro/.

[12] Michael Eddington. 2020. Peach. https://peachtech.gitlab.io/peach-fuzzer-community/.

[13] Kaiwen Shen et al. 2022. Code Package of Hdiff. https://github.com/mo-xiaoxi/HDiff.

[14] Roy T. Fielding et al. 2014. Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing. https://datatracker.ietf.org/doc/html/rfc7230.

[15] Xiaotao Feng, Ruoxi Sun, Xiaogang Zhu, Minhui Xue, Sheng Wen, Dongxi Liu, Surya Nepal, and Yang Xiang. 2021. Snipuzz: Black-box Fuzzing of IoT Firmware via Message Snippet Inference. In Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (Virtual Event, Republic of Korea) (CCS '21). Association for Computing Machinery, New York, NY, USA, 337–350. doi:10.1145/3460120.3484543

[16] Ivan Fratric. 2016. NETAFL. https://github.com/intelpt/winafl-intelpt.

[17] Matheus E. Garbelini, Vaibhav Bedi, Sudipta Chattopadhyay, Sumei Sun, and Ernest Kurniawan. 2022. BrakTooth: Causing Havoc on Bluetooth Link Manager via Directed Fuzzing. In 31st USENIX Security Symposium (USENIX Security 22). USENIX Association, Boston, MA, 1025–1042. https://www.usenix.org/conference/usenixsecurity22/presentation/garbelini

[18] Ben Hawkes. 2022. 0day In the Wild. https://googleprojectzero.blogspot.com/p/0day.html.

[19] Siva Kesava Reddy Kakarla, Ryan Beckett, Todd Millstein, and George Varghese. 2022. SCALE: Automatically Finding RFC Compliance Bugs in DNS Nameservers. In 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22). USENIX Association, 307–323.

[20] Wei-Che Kao, Yen-Chia Chen, Yu-Sheng Lin, Yu-Cheng Yang, Chi-Yu Li, and Chun-Ying Huang. 2025. {BLuEMan}: A Stateful Simulation-based Fuzzing Framework for {Open-Source}{RTOS} Bluetooth Low Energy Protocol Stacks. In 34th USENIX Security Symposium (USENIX Security 25). 6259–6278.

[21] Stephen Kent and Karen Seo. 2005. Security architecture for the internet protocol.

[22] Zhengxiong Luo, Junze Yu, Feilong Zuo, Jianzhong Liu, Yu Jiang, Ting Chen, Abhik Roychoudhury, and Jiaguang Sun. 2023. Bleem: Packet Sequence Oriented Fuzzing for Protocol Implementations. https://www.usenix.org/conference/usenixsecurity23/presentation/luo-zhengxiong. In 32nd USENIX Security Symposium (USENIX Security 23). USENIX Association, Anaheim, CA, 4481–4498.

[23] Xiaoyue Ma, Lannan Luo, and Qiang Zeng. 2024. From One Thousand Pages of Specification to Unveiling Hidden Bugs: Large Language Model Assisted Fuzzing of Matter {IoT} Devices. In 33rd USENIX Security Symposium (USENIX Security 24). 4783–4800.

[24] Ruijie Meng, Martin Mirchev, Marcel Böhme, and Abhik Roychoudhury. 2024. Large language model guided protocol fuzzing. In Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS).

[25] Roberto Natella. 2022. Stateafl: Greybox fuzzing for stateful network servers. Empirical Software Engineering 27, 7 (2022), 191.

[26] OpenAI. 2023. GPT-4o. https://openai.com/index/hello-gpt-4o/.

[27] OpenAI. 2025. GPT-5. https://openai.com/index/introducing-gpt-5/.

[28] Maria Leonor Pacheco, Max von Hippel, Ben Weintraub, Dan Goldwasser, and Cristina Nita-Rotaru. 2022. Automated attack synthesis by extracting finite state machines from protocol specification documents. In 2022 IEEE Symposium on Security and Privacy (SP). IEEE, 51–68.

[29] Joshua Pereyda. [n. d.]. boofuzz: Network Protocol Fuzzing for Humans. https://boofuzz.readthedocs.io/en/stable/index.html.

[30] Van-Thuan Pham, Marcel Böhme, and Abhik Roychoudhury. 2020. AFLNet: a greybox fuzzer for network protocols. In 2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST). IEEE, 460–465.

[31] Van-Thuan Pham, Marcel Böhme, and Abhik Roychoudhury. 2020. AFLNET: A Greybox Fuzzer for Network Protocols. In 2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST). 460–465. doi:10.1109/ICST46399.2020.00062

[32] TLSFuzzer Project. 2025. TLSFuzzer. https://github.com/tlsfuzzer/tlsfuzzer.

[33] Lili Quan, Qianyu Guo, Hongxu Chen, Xiaofei Xie, Xiaohong Li, Yang Liu, and Jing Hu. 2020. SADT: Syntax-Aware Differential Testing of Certificate Validation in SSL/TLS Implementations. In 2020 35th IEEE/ACM International Conference on Automated Software Engineering (ASE). 524–535.

[34] Eric Rescorla. 2018. The transport layer security (TLS) protocol version 1.3. https://www.rfc-editor.org/rfc/rfc8446.

[35] Sergej Schumilo, Cornelius Aschermann, Andrea Jemmett, Ali Abbasi, and Thorsten Holz. 2022. Nyx-net: network fuzzing with incremental snapshots. In Proceedings of the Seventeenth European Conference on Computer Systems (Rennes, France) (EuroSys '22). Association for Computing Machinery, New York, NY, USA, 166–180. doi:10.1145/3492321.3519591

[36] Kaiwen Shen, Jianyu Lu, Yaru Yang, Jianjun Chen, Mingming Zhang, Haixin Duan, Jia Zhang, and Xiaofeng Zheng. 2022. Hdiff: A semi-automatic framework for discovering semantic gap attack in http implementations. In 2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). IEEE, 1–13.

[37] Cong Tian, Chu Chen, Zhenhua Duan, and Liang Zhao. 2019. Differential testing of certificate validation in SSL/TLS implementations: An RFC-guided approach. ACM Transactions on Software Engineering and Methodology (TOSEM) 28, 4 (2019), 1–37.

[38] Jincheng Wang, Le Yu, and Xiapu Luo. 2024. Llmif: Augmented large language model for fuzzing iot devices. In 2024 IEEE Symposium on Security and Privacy (SP). IEEE Computer Society, 196–196.

[39] Wireshark Project. 2025. Wireshark. https://www.wireshark.org/about.html.

[40] Xiaohan Zhang, Cen Zhang, Xinghua Li, Zhengjie Du, Bing Mao, Yuekang Li, Yaowen Zheng, Yeting Li, Li Pan, Yang Liu, et al. 2024. A survey of protocol fuzzing. Comput. Surveys 57, 2 (2024), 1–36.

[41] Yong-Hao Zou, Jia-Ju Bai, Jielong Zhou, Jianfeng Tan, Chenggang Qin, and Shi-Min Hu. 2021. {TCP-Fuzz}: Detecting memory and semantic bugs in {TCP} stacks with fuzzing. In 2021 USENIX Annual Technical Conference (USENIX ATC 21). 489–502.

Appendix

In the appendix, we provide the supporting materials for this paper, including the prompt design, the message types involved in each protocol, the response mapping mechanism, and detailed information on the identified potential vulnerabilities.

A. Prompt Design for SemFuzz

The prompt designs of our method are illustrated in Fig. 4, Fig. 5, Fig. 6, and Fig. 7, which correspond to specification identification, semantic rule completion, mutation strategy generation, and action sequence generation, respectively.

B. Provided Message Types

As shown in Table 8, in our experiment, we provided key message types for each protocol. These include 2 types of DNS messages, 10

Table 8: The chosen message type

Protocol	Message Type	Count
DNS	① DNS Query, ② DNS Response	2
IPv6	① IPv6 header with Authentication Header, ② IPv6 header with ICMPv6, ③ IPv6 header with TCP, ④ IPv6 header with UDP, ⑤ IPv6 header with Destination Options Header, ⑥ IPv6 header with Encapsulating Security Payload Header, ⑦ IPv6 header with Fragment Header, ⑧ IPv6 header with Hop-by-hop Options Header, ⑨ IPv6 header (with No Next header), ⑩ IPv6 header with Routing Header	10
TLS 1.3	① ClientHello with Application Layer Protocol Negotiation Extension, ② ClientHello with Application Settings Extension, ③ ClientHello with Compress Certificate Extension, ④ ClientHello with Ec Point Formats Extension, ⑤ ClientHello with Encrypted Client Hello Extension, ⑥ ClientHello with Extended Master Secret Extension, ⑦ ClientHello with Key Share Extension, ⑧ ClientHello with No Extension, ⑨ ClientHello with Pre Shared Key Extension, ⑩ ClientHello with Psk Key Exchange Modes Extension, ⑪ ClientHello with Renegotiation Info Extension, ⑫ ClientHello with Reserved Extension, ⑬ ClientHello with Server Name Extension, ⑭ ClientHello with Session Ticket Extension, ⑮ ClientHello with Signature Algorithms Extension, ⑯ ClientHello with Signed Certificate Timestamp Extension, ⑰ ClientHello with Status Request Extension, ⑱ ClientHello with Supported Groups Extension, ⑲ ClientHello with Supported Versions Extension	19
HTTP/1.1	① http request with Host header, ② http request with request.line header, ③ http request with User-Agent header, ④ http request with Accept header, ⑤ http request with Accept-Encoding header, ⑥ http request with Authorization header, ⑦ http request with http.cache_control header, ⑧ http request with Content-Type header, ⑨ http request with Content-Length header, ⑩ http request with Date header, ⑪ http request with Last-Modified header, ⑫ http request with Server header, ⑬ http request with Location header	13

```
@Persona: You are a protocol security expert. Your task is to identify protocol specification
        requirements from a given text.
@Instructions:
    @InputVariable:
        ${text}$
        ${message_type_list}$
    @Command Based on the provided message type list, identify all specific message types
        mentioned in the text.
    @Command For each identified message type, extract the specific content of the
        specification requirement from the text.
    @OutputVariable:
        ${specification_requirements}$
    @Format ...
    @Examples ...
```

Figure 4: Prompt for specification identification

```
@Persona: You are a protocol security expert. Your goal is to translate the specification
        requirements into structured rules.
@Instructions:
    @InputVariable:
        ${specification_requirement}$
        ${message_structure}$
    @Command Identify the relevant message field associated with the specification
        requirement based on the message structure.
    @Command Analyze the specification requirement to identify both construction and
        processing rules.
    @Command Identify the roles in different types of rules.
    @Command Infer the missing rule in the specification requirement to complete the
        structured semantic rule.
    @OutputVariable:
        ${semantic_rule}$
    @Format ...
    @Examples ...
```

Figure 5: Prompt for semantic rule completion

types of IPv6 messages, 19 types of TLS 1.3 messages, and 13 types of HTTP/1.1 messages.

C. Response Mapping Mechanism

SemFuzz defines a protocol-specific response mapping mechanism that maps actual and expected responses into "normal" or "error" categories. The details are as follows:

For DNS, a response is classified as normal when the server returns data corresponding exactly to the queried domain. If the response includes unrelated domain records or indicates a query processing error, it is treated as an error case. For IPv6, normal behavior is identified when the receiver replies with an ICMPv6 Echo Reply, indicating successful packet handling. Error cases include the return of ICMPv6 error messages, such as Time Exceeded or Parameter Problem, or the absence of a response. For TLS 1.3, the exchange is considered normal when the server continues the handshake after receiving a ClientHello message, producing messages such as ServerHello. Failure to respond or to proceed with the handshake is considered an error (e.g. Alert). For HTTP, a normal case involves the server returning a success status code such as 200 OK, confirming proper request handling. Error cases include responses with failure status codes, such as 400 or 500, or a complete lack of response. This mapping mechanism enables SemFuzz to consistently interpret implementation behavior across different protocols and detect semantic vulnerabilities.

```
@Persona: You are a protocol security expert.Your task is to generate rule violations based on
        the construction rules, and
then develop precise mutation strategies for those violations.
@Terminology:
    @Term Expected response List: "Normal feedback": "The server accepted the ClientHello
        message and proceeded with the handshake...", "Error feedback": "The server did not
        respond to the ClientHello message."...
@Instructions:
    @InputVariable:
        ${semantic_rule}$
    @Command Analyze the semantics of the construction rule to generate the direct violation
        of the construction rule.
    @Command Generate specific mutation strategies for the violation of the construction rule.
    @Command Based on the processing rules, select the appropriate expected behavior from
        the expected behavior list for each test strategies.
    @OutputVariable:
        ${mutation_strategies}$
    @Format ...
    @Examples ...
```

Figure 6: Prompt for mutation strategy generator

```
@Persona: You are a protocol security expert.Your task is to generate precise action sequences
        by analyzing the provided mutation strategies and message structures.
@Terminology:
    @Term Actions instructions:
        update():...; remove()...; add()...
@Instructions:
    @InputVariable:
        ${mutation_strategy}$
        ${message_structure}$
    @Command Analyze each mutation strategy to determine the necessary modification to the
        message.
    @Command For each necessary modification, choose the corresponding actions and
        complete its parameters.
    @Command Ensure that the sequence of actions maintains the integrity of the message
        structure...
    @OutputVariable:
        ${action_sequence}$
    @Format ...
    @Examples ...
```

Figure 7: Prompt for action sequence generation

Table 9: Detected potential vulnerability in Dns.exe, Tcpip.sys, and Schannel.dll

Target	Cause
Dns.exe	The vulnerability was exploited using a mutation strategy where an additional domain, not requested by the server, was inserted into the response message. This mutation resulted in a response containing not only the IP address corresponding to the server's requested domain but also the IP address linked to a not requested domain. The expected behavior was that the server would send an error feedback. However, during processing, the server only verified the presence of its requested domain within the response. If found, the server accepted the entire response and cached it, thereby unintentionally storing the IP address associated with the not requested domain. This oversight led to cache poisoning, as the server's cache was contaminated with invalid domain-IP mappings.
	The false positive was detected due to an atomic action sequence error, where the test case generator mistakenly modified "www.example.com" to "example.com" (instead of the correct "WWW.EXAMPLE.COM"). It was expected that the server would reject this query and return an error. However, in reality, the server not only returned an IP address for subdomains of "example.com" but also for "example.com" itself. This inconsistency between the expected and actual behavior resulted in the false positive.
Tcpip.sys	The vulnerability was identified when the mutation strategy added multiple AH headers to the original packet, ultimately resulting in an unusually long IPv6 message that was expected to be accepted by the server and server would send a normal feedback. However, when the server attempted to process the message, it crashed, causing a discrepancy between the next strategy expected which is a normal feedback and actual results and meaning to send an error feedback. Expert analysis confirmed that the crash was triggered by the inability of the IPv6 node to handle the oversized packet with a special payload field.
	The vulnerability was discovered when the mutation strategy placed the ESP header as part of the per-fragment, rather than as part of the fragment itself, leading to the generation of an IPv6 packet with the ESP header preceding the Fragment Header. This packet was expected to be rejected by the server and server would send an error feedback. However, during processing, the server crashed, causing a deviation from the next strategy expected outcome whose expected feedback is normal feedback. Expert analysis confirmed that the crash occurred due to a buffer overflow vulnerability triggered when the server attempted to reassemble the fragmented packet.
	The false positive was identified due to an error in the mutation strategy when generating the action sequence, which led to an incorrect output for the field corresponding to the modified message. This error resulted in the generation of a valid response message containing extra data, which was expected to be rejected by the server and server would send a normal feedback. However, when processing such messages, the server typically discarded the extraneous data and responded an error feedback, causing a discrepancy between the expected and actual outcomes and ultimately leading to the false positive.
	The vulnerability was identified due to the mutated message having an Option Type value exceeding 0x80. According to the RFC, the server should discard the packet and reply with an ICMP message with Code 2. However, the server did not immediately discard the packet. Instead, it continued to process the subsequent packet headers, which led to a buffer overflow vulnerability and caused the server to crash, resulting in a timeout in response.
Schannel.dll	The vulnerability was identified due to a mutation strategy that placed the PSK extension before the supported_versions field, resulting in a message where the PSK was not the final extension. The expected behavior was for the server to reject this message with an error feedback. However, upon processing the message, the server detected an anomaly and responded with a normal feedback, which deviated from the expected outcome. Upon expert investigation, it was found that the server performed an identity check on the PSK, which caused the constructed message to fail validation and trigger the normal feedback. However, the server did not check whether the PSK was the last extension, which should have led to a rejection with the error feedback if the PSK were not the final extension.
	The vulnerability was discovered in a manner similar to the previous one, but it applies to a scenario involving multiple servers with ticket-based verification, as opposed to the single-server binder validation scenario described earlier.
	The false positive was identified when the mutation strategy caused the client to send application data prior to the certificate exchange for authentication, with the expectation that the server would reject the packet and send an error feedback. However, the experimental setup involved a server that did not perform client authentication, allowing the server to continue communication with the client and successfully send a normal feedback.

Table 10: Detected potential vulnerability in Http.sys and Openssl

Target	Cause
Http.sys	The vulnerability was discovered when the mutation strategy introduced an extra whitespace between the Content-Length header and the colon in the original packet before sending it to the server. The expectation was that the server would reject the malformed request and return an error response. However, instead of rejecting it, the server accepted the packet and responded with a normal response. After expert verification, this behavior was confirmed to be exploitable for HTTP Request Smuggling.
	The vulnerability was discovered when the mutation strategy injected a list of malformed or unknown content-coding tokens into the Accept-Encoding header. While the RFC requires the server to ignore unsupported encodings, the http.sys driver failed to properly validate the object during the parsing of these invalid tokens. This oversight triggered a UAF condition, causing the kernel to crash and preventing a normal HTTP response.
	The vulnerability was identified because the Trailer header fields generated by mutate contained multiple random values without actual meaning, and the request utilized chunked transfer encoding. According to the RFC, the server should correctly handle this packet and normally reply to the HTTP request. However, the reality is that the server crashed due to a buffer overflow vulnerability when processing these random trailer values, resulting in a timeout for the reply.
	The vulnerability is triggered because the length of the ServiceName generated by mutate in the request is very long. After accepting such a packet, the server should reply that the packet is normal. However, in reality, the server crashes due to an integer overflow vulnerability caused by incorrect length validation of the original ServiceName input, resulting in a timeout for the reply which is an error reply.
Openssl	This false positive was triggered by the premature transmission of application data before the certificate exchange. While the fuzzer anticipated a rejection, the server accepted the data simply because client authentication was disabled in the experimental configuration.
	Due to a mutation strategy that inserted the PSK extension before the supported_versions field, the generated message had the PSK not as the final extension. The server was expected to reject this message and return an error feedback; however, it instead responded with normal feedback. However, since the vulnerability is difficult to exploit, the developer does not consider it a vulnerability.
	When the mutation strategy added a new field — the 0xffff PSK key exchange mode — into the original packet and sent it to the server, the server was expected to reject the packet and return an error feedback. However, the server accepted the packet and responded with normal feedback. However, since the vulnerability is difficult to exploit, the developer does not consider it a vulnerability.

D. Detected Potential Vulnerabilities

As shown in Table 9 and Table 10, we summarize the 16 potential vulnerabilities, among which 10 have been confirmed, including 5 previously unknown vulnerabilities.

Dns.exe. Two potential vulnerabilities were identified. One is a new cache pollution vulnerability caused by the server failing to verify the presence of extra domain names in responses. The second, a false positive, resulted from the mutation strategy that produced a valid domain under the server's authority, leading to normal processing.

Tcpip.sys. Four potential vulnerabilities were identified. Specifically, we identified a resource exhaustion vulnerability triggered by the processing of oversized packets with excessive `Authentication` Headers. Furthermore, two memory vulnerabilities were detected: a buffer overflow resulting from the incorrect placement of `ESP` headers during packet fragmentation, and an integer overflow occurring within the parsing logic of the `Destination Options` header. The last one is a false positive, where the server discarded messages containing invalid extra data and continued normal operation.

Http.sys. A total of four vulnerabilities were detected. The first involves improper handling of whitespace in the `Content-Length` header, enabling HTTP Request Smuggling. The next two include a UAF caused by parsing the `Accept-Encoding` field and a buffer overflow triggered by malformed HTTP `Trailers`. The final vulnerability stems from incorrect length validation of the `ServiceName`, causing an integer overflow.

Schannel.dll. Three potential vulnerabilities were detected. Two of them are previously unknown issues that have been assigned CVEs. Both are related to incorrect positioning or omission of the `PSK` extension in the `ClientHello` message, allowing malformed handshakes to succeed. The remaining one was a false positive, where the client sent application data before authentication when client authentication was disabled, but it caused no harm.

Openssl. Three potential vulnerabilities were detected. The first involves the client sending application data prematurely, which was misidentified as a vulnerability because client authentication was disabled. The second is an incorrect placement of the `PSK` extension, where the server failed to reject the abnormal handshake as required by the RFC. The third is the absence of the `PSK` key exchange mode extension, causing the server to accept an invalid request. Since the latter two potential vulnerabilities are difficult to exploit in practice, they are not considered real vulnerabilities.